

A Comparative Analysis of Different ML Models for Wearable Devices Centric Human Activity Recognition

Namia Jalal Zara
Electrical and Computer Engineering
North South University
Dhaka, Bangladesh
namia.zara@northsouth.edu

Jarin Tasnim
Electrical and Computer Engineering
North South University
Dhaka, Bangladesh
jarin.tasnim10@northsouth.edu

Rashiqur Rahman Rifat
Electrical and Computer Engineering
North South University
Dhaka, Bangladesh
rashiqur.rifat@northsouth.edu

Md. Baker
Electrical and Computer Engineering
North South University
Dhaka, Bangladesh
md.baker@northsouth.edu

Abstract—This paper presents a comprehensive study on applying machine learning models for human activity recognition. The increasing number of wearable devices and the growing interest in health monitoring and smart environments have fueled the demand for robust and accurate systems that automatically recognize human activities. This study explores and compares the performance of different machine-learning models in human activity recognition. We aim to identify the most effective ML model to accurately classify and predict human activities. We evaluate various models, including traditional classifiers such as Decision Tree, Random Forest, K-Nearest Neighbor, Logistic Regression, AdaBoost algorithm, and XGBoost algorithm. All models have been evaluated using the publicly available dataset "Human Activity Recognition with Smartphones", which captures daily activities: Lying, Standing, Sitting, Walking, Walking Upstairs, and Walking Downstairs. Among all the models we applied, XG Boost gave the highest accuracy of 99.52% based on precision (100%), recall (100%), and F1 score (100%). We performed hyperparameter tuning on the ML models to attain the best accuracy.

Keywords— HAR, KNN, SVM, XGBoost, LR, AdaBoost, EDA, LIME, Decision Tree, Random Forest, Explainable AI, GCD, hyperparameter.

I. INTRODUCTION

A specialized computer science and engineering area called "human activity recognition" (HAR) aims to automatically identify and classify human behaviours from wearable devices. The method of analyzing human motion with computer vision and other technologies is known as HAR. Sensor-recorded actions, gestures, or behaviours can be understood as human motion. HAR can be carried out via sensors, smartphones, or pictures [1]. The present models for activity recognition, such as recurrent, convolutional, or hybrid models, have difficulty extracting spatiotemporal context from the feature space of sensor reading sequences [2]. Since cooperation between robots and people needs artificial agents to recognize complicated environmental cues, HAR is a crucial problem in human-robot interaction [1]. Mainly, we will use machine learning models, such as Random Forest, Decision Tree, KNN, Logistic Regression, Ada Boost, XGBoost, etc., to evaluate the activity of a human by wearable devices. Human Activity Recognition provides advantages such as personalized healthcare

monitoring and fitness training, adaptive smart environments, enhanced security through intruder detection, and improved industrial safety by identifying hazardous activities.

Mahajan and colleagues [3] studied human activity recognition from videos using K-mean and ANN. They used a private dataset, and they made it using different devices. Most of those videos had noises, so they preprocessed activity videos. They also used Mean Squared Error (MSE) to evaluate the performances. By using Artificial Neural Networks, they had an accuracy of 98.24%.

Gumaei and colleagues [4] studied human activity from body sensors using deep learning models like RNN (GPU, SRU). They used the MHEALTH dataset, which is a public dataset. Body motion and different sins of 10 subjects were used in this dataset. They used precision and F1-score to evaluate the performances. They had an accuracy of 99.80% for deep SRUs-GRUs and 94.145 for deep SRUs.

Liu et al. [5] studied new methodologies for top-to-bottom-based human movement examination and demonstrated the viability of 3D human action analysis. Large-scale human activity recognition datasets, comprising 56,000 video tests and 4 million edges gathered from 40 participants, were provided in this paper (NTU RGB+D dataset, SBU Interaction dataset, UT-Kinect dataset, Berkeley MHAD dataset). There are sixty activity classes available, from daily to shared to wellness-focused. For the Berkeley MHAD dataset, they achieved 100% accuracy using HBRNN techniques.

Calatrava-Nicolas et al. [6] studied human activity recognition (HAR) using a light residual network for wearable sensor data. Their goal of using Light Residual Network (ResNet) is to perform tasks such as modifying ResNet18 and reducing the complexity of the model and parameters. They used the UCI-HAR dataset, which contains different activities, and they focused on six activities. They compared their work with previous work F1-Score, accuracy, and number of parameters, and they got the best accuracy, 97.6%, for their model.

Ghazal and his team [7] studied human activity recognition using 2D skeleton data and supervised machine learning. Two publicly available datasets were used for the

performance evaluation. They evaluated the performance of various classifiers, including Feed-forward, KNNs, SVM, Discriminant analysis, and Naive Bayes. Naive Bayes achieved 90% accuracy in the first set of experiments, followed by SVM at 76%. Linear Discriminant Analysis (LDA) excelled in classifying falls, while Naive Bayes and KNN achieved 100% recall for standing. After data normalization, all classifiers showed increased accuracy, with KNN reaching 98%.

Pathan and his colleagues [8] studied machine learning-based human activity recognition during physical exercise using wavelet packet transform of PPG and Inertial Sensors data. They used publicly available PPG data collected from the physionet. The study used an 8-second time series window with 75% overlap for frame-by-frame physiological and inertial data analysis, using Wavelet Packet Transform for feature extraction. The Random Forest classifier achieved 97.8% accuracy in recognizing human activities, outperforming a previous SVM-CNN-based method, demonstrating efficiency and improved performance.

Leema and colleagues [9] studied human activity recognition using machine learning. They used a private dataset, and they made it using different devices. To analyze human activity, they used a variety of machine classification algorithms, including Random Forest, kNN, Neural Network, Logistic Regression, Stochastic Gradient Descent, and Naïve Bayes. Test accuracy is 99.55%.

Manoj [10] and associates used PCA and LDA to study human activity recognition. They used the UCI ML dataset. The findings indicate that LDA outperforms PCA. The kernel-SVM's accuracy has increased to 96.23% from 95.39%. Naïve Bayes has shown 96.78% accuracy in dimensionality reduction. While dimensionality has improved model performance, simple datasets have shown low accuracy.

We can infer from the paragraphs above that there has been a lot of study on human activity recognition. Still, the number of comparisons between the machine learning models could be higher. These researchers made accurate predictions by utilizing a range of machine-learning approaches. Most of these studies examined the predictions made by machine learning models using various publicly and privately available datasets without using any comprehensible artificial intelligence techniques. These studies have thus recommended a variety of open-source and original datasets together with explainable AI methods.

Our work used several machine learning models to predict the possible Human activity by wearable devices and compare the accuracy of the machine learning models. We used a dataset from Kaggle, where they took 563 features to identify whether a person is walking, lying, standing, sitting, walking upstairs, or walking downstairs. We used machine learning models in our project, such as decision tree, random forest, KNN, logistic regression, etc. We did hyperparameter tuning for various ML models by using GridSearchCV and RandomSearchCV. We improved the validity of the research. We conducted by implementing the Explainable AI library LIME to evaluate the outcome and identify the main contributing components that impact the prediction process.

Below is the structure of this paper: A description of the dataset, dataset preprocessing, and machine learning models and architecture applied in the study are provided in Section II, which also discusses the proposed system. Section III presents the outcomes and a discussion of the results using machine learning models. The comparative analysis of this research is shown in Section IV. And in Section V, we conclude the paper and future works.

II. PROPOSED SYSTEM

In this part, we are going to describe our dataset, preprocessing system, and different models that we have used in our work.

A. Dataset

The trials involved twenty-eight participants, ranging in age from 19 to 48. Each participant performed six distinct tasks while wearing a smartphone (Samsung Galaxy S II) around their waist: walking, going upstairs, walking downstairs, sitting, standing, and lying down. We used its built-in accelerometer and gyroscope to capture 3-axial angular velocity and 3-axial linear acceleration at a constant rate of 50Hz. The trials were videotaped to identify the data manually. The gathered dataset was randomly split into two sets, with 30% of the participants selected to provide test data and 70% of the volunteers selected to offer training data. The accelerometer and gyroscope sensor signals were pre-processed using noise filters before being sampled in fixed-width sliding windows for 2.56 seconds (128 readings/window) with a 50% overlap. The sensor acceleration signal, which includes both body motion and gravitational components, was split into two categories using a Butterworth low-pass filter: body acceleration and gravity. Because the gravitational force is thought to be made up only of low-frequency components, a 0.3 Hz cutoff frequency filter was used. From each window, a vector of features was obtained by calculating variables in the frequency and temporal domains. The dataset's features table is shown in Table I.

TABLE I. FEATURES TABLE OF DATASET

Attribute	Data Type	Mean Value
tBodyAcc-mean()-X	float	0.274488
tBodyAcc-mean()-Y	float	-0.017695
tBodyAcc-mean()-Z	float	-0.109141
tBodyAcc-std()-X	float	-0.605438
tBodyAcc-std()-Y	float	-0.510938
tBodyAcc-std()-Z	float	-0.604754
tBodyAcc-mad()-X	float	-0.630512
tBodyAcc-mad()-Y	float	-0.526907
tBodyAcc-mad()-Z	float	-0.606150
fBodyBodyGyroJerkMag-kurtosis()	float	-0.307009
angle(tBodyAccMean,gravity)	float	-0.625294

angle(tBodyAccJerkMean,gravityMean)	float	0.008684
angle(tBodyGyroMean,gravityMean)	float	0.002186
angle(tBodyGyroJerkMean,gravityMean)	float	0.008726
angle(X,gravityMean)	float	-0.489547
angle(Y,gravityMean)	float	0.058593
angle(Z,gravityMean)	float	-0.056515
subject	int	17.413085
Activity	object	—

According to Table I, we see various features, data types and mean value. We have more than five hundred features but here we show nineteen features and their details. Most of them are float values. Some of the features are integer and object type.

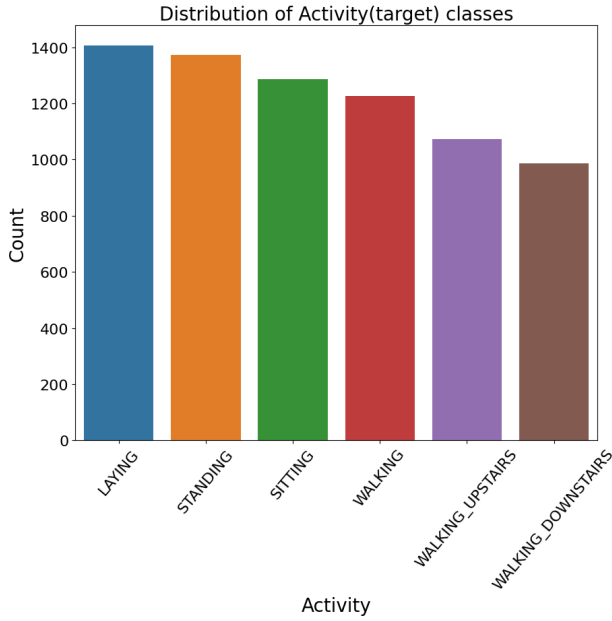


Fig. 1. Exploratory Data Analysis.

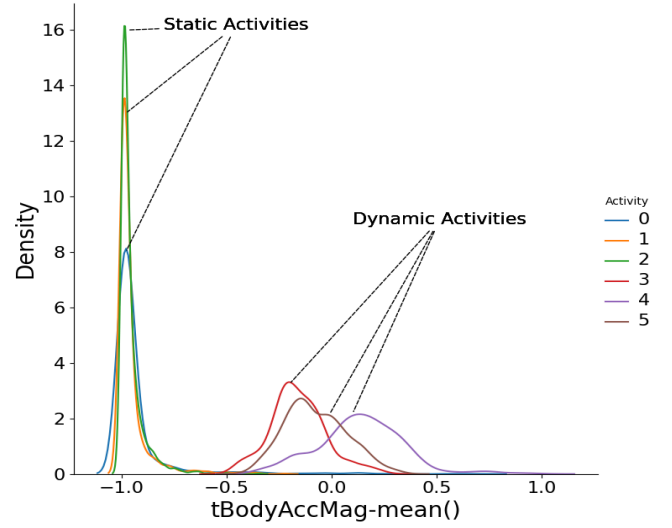


Fig. 2. Exploratory Data Analysis.

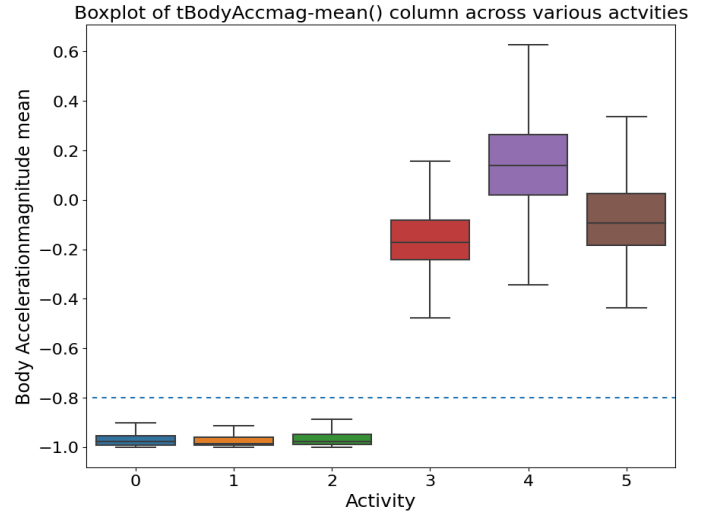


Fig. 3. Exploratory Data Analysis.

According to Fig-1, we can see all the activities of target classes like laying, standing, walking, sitting, etc. According to Fig-2, we see Exploratory Data Analysis. Here, we see two types of activities: dynamic and static. According to Fig-3, we see the box plot. The EDA illustrates the dataset is not imbalanced or has that much outlier. The dataset was separated into two sections: training (80%) and testing (20%). The testing set assisted in evaluating the performance of our trained models.

B. Dataset Preprocessing

In our dataset, we use six different models with different models having their barriers. For example, the decision tree model cannot work on null values. That's why we need to pre-process our dataset.

Our dataset is a balanced dataset with no null values or missing values. It also has no duplicate values. In the beginning, we checked all of this. We also know that models do not work on categorical values. It only understands numerical values. So, we use label encoding to transfer the categorical value to the numerical value. In our dataset, our target class, which means the "Activity", has a categorical

value, which we changed into numerical values 0 to 5 using label encoding because we have 6 classes in our dataset.

We also used Minmaxscaler as feature scaling. At first, we used feature scaling, and then we placed our data into the train and tested it with a ratio of 8:2.

C. Preparing Machine Learning Algorithms

1) *GridSearchCV*: Grid search is a parameter tuning technique that systematically builds and examines a model for every set of algorithm parameters in a grid. It provides the optimal model parameters by fine-tuning the hyperparameter. It improves a model's performance but takes a long time because it looks through every possible combination of the specified grid components.

2) *RandomizedSearchCV*: Unlike *GridSearchCV*, *RandomizedSearchCV* is a machine-learning method for tuning hyperparameter combinations randomly from predetermined ranges. This method preserves the search for the best model configurations while reducing computing expenses. *RandomizedSearchCV* effectively searches hyperparameter spaces by arbitrarily choosing combinations, offering a decent compromise between exploitation and exploration. This approach offers a more useful substitute for thorough grid searches when the search space is large. Through the use of a resource-efficient, randomized search procedure, *RandomizedSearchCV* improves model performance and aids in the optimization of machine learning models.

3) *Explainable AI*: Explainable artificial intelligence is a technique that is helpful for people who are not experts in machine learning or need help understanding the output of algorithms. This process is normally called "Explainable AI." It is mainly used to explain any AI model. In our work, we applied LIME, which is very user-friendly.

4) *LIME*: "Local Interpretable Model-agnostic Explanations" is known as "LIME". The LIME approach uses a locally approximable interpretable model to faithfully explain the predictions of any classifier or regressor. In our work, we used it to understand our models.

D. Machine Learning Models

1) *Decision tree*: A decision tree which is a widely used machine learning technique because it can classify dependent variables into different classes. It scans the data and identifies the most important independent factors, considering several possible variables to make predictions [11]. A decision tree is a supervised classification method. That is a simple structure where evaluation results are displayed in terminal nodes, and checks for one or more functions are displayed in non-terminal nodes. Previous research on human action detection using decision tree classifiers has produced encouraging results.

2) *Random Forest*: The Random Forest ensemble understanding technique creates many decision trees during training. A subset of the bagging technique called the RF performs better when noise and poor discriminating data are not affected by parameter initialization overfitting [12]. Breiman's study shows that the RF's generalization error converges as the number of trees rises, a feature lacking in most other classifiers [17]. With its excellent accuracy, ability to handle big datasets with plenty of features, and ability to provide feature-importance insights, Random

Forest is very useful for classification and regression applications.

3) *Logistic Regression*: This fundamental machine-learning technique is used in binary classification [13]. Despite its name, its purpose is to calculate the probability that an instance will fall into a given class. The program simulates this chance using the logistic function, which adjusts the input data linearly. Logistic regression is a versatile tool that may be applied in many different settings because of its effectiveness, interpretability, and simplicity. Predictions are based on a threshold, and coefficients are optimized by maximum likelihood estimation. In many machine-learning contexts, the fundamental logistic regression method is utilized in finance and medicine to predict binary outcomes.

4) *AdaBoost*: AdaBoost, which stands for Adaptive Boosting, is an ensemble learning algorithm for classification and regression [14]. AdaBoost is a learning algorithm that combines weak learners (such as SVC) to form a strong classifier. It is especially effective with weak learners or models that are better than random chance. It constructs a series of weak learners to correct errors and iteratively focuses on misclassified instances, assigning higher weights to them. A weighted average of these learners is used to make the final prediction, resulting in a robust and accurate model.

5) *KNN*: K-Nearest Neighbors (KNN) is a simple machine-learning algorithm for classification and regression [15]. It can generate predictions by comparing newly collected data points to labelled data points in the training set. The algorithm determines the K nearest neighbors using weighted averaging or majority voting, computes distances, and forecasts the class label for classification or regression value. Depending on the parameter K selection, KNN is an instance-based, non-parametric model.

6) *XGBoost*: XGBoost is a distributed gradient boosting library that has been optimized to be incredibly effective, versatile, and lightweight. It sequentially builds a series of decision trees, each correcting errors made by the preceding ones. The model is trained by minimizing a specific objective function, incorporating regularization terms for enhanced predictive performance. XGBoost employs a novel regularization term, the "eXtreme Gradient," that helps control model complexity. It is widely used for classification, regression, and ranking tasks due to its efficiency, speed, and ability to handle large datasets. The XGBoost model has been developed to optimize multimode fibre architecture and to diagnose cause-aware network failures with improved estimate and precision. XGBoost has become a go-to algorithm in machine learning competitions and is known for achieving state-of-the-art results [16].

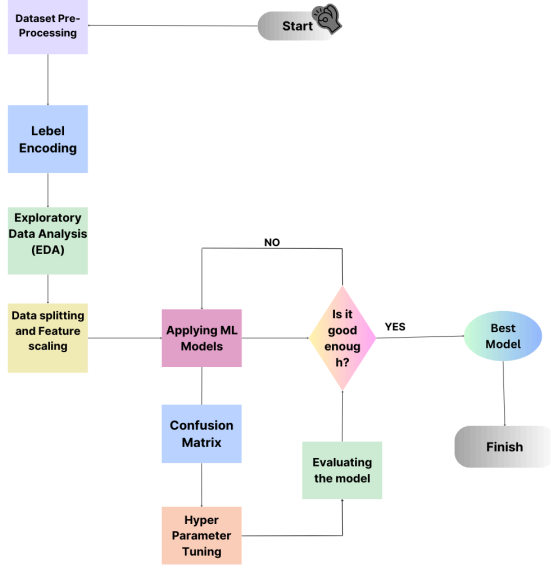


Fig. 4: Working sequences of the proposed HAR system.

In Fig. 4, we showed the workflow of our project. In this project, we took a public dataset to apply our machine learning model. But before applying the ML models, we pre-processed the dataset. After that, we did label encoding on the dataset. Then, we applied exploratory data analysis (EDA) to analyze the data. After EDA, we split the data into the train (80%), test (20%) data, and implied feature scaling. After doing all these processes, we applied machine learning models and found the matrix and classification report. For better accuracy of the models, we applied hyperparameter tuning and evaluated the model. For comparing the ML models, we applied several ML models in our project so that we could compare the models with the best accuracy, whether the model was overfitted or underfit. We discussed the comparison of the ML models that we applied in our Result and Discussion section.

III. RESULTS AND DISCUSSION

This section of the paper discusses the results of all different Machine Learning algorithms for Human Activity Recognition. We have applied six Machine Learning models in the dataset "Human Activity Recognition with Smartphones" from Kaggle. In our dataset, there are 563 columns and 7352 rows of attributes. We split the data into train (80%) and test (20%) data. In 563 columns, we used one column as our target variable column, which has 6 classes and a multiclass problem, and the other 562 columns are features or predictors. We evaluate the performance metrics from the model, including accuracy, Recall, F1 score, and precision. These matrices can be calculated through the following equations:

$$Accuracy = \frac{\text{correct predictions}}{\text{all predictions}} = \frac{TP+TN}{TP+TN+FP+FN} \quad (1)$$

$$Precision_c = \frac{TP_c}{TP_c + FP_c} \quad (2)$$

$$Recall_c = \frac{TP_c}{TP_c + FN_c} \quad (3)$$

$$F1\ Score_c = \frac{2 \times (Precision_c \times Recall_c)}{Precision_c + Recall_c} \quad (4)$$

where TP = True Positives, TN = True Negatives, FP = False Positives, FN = False Negatives, c = class label (1 to 6), TP_c = True Positives for class c, FN_c = False Negatives for class c, FP_c = False Positives for class c, Precision_c = Precision for class c, Recall_c = Recall for class c, F1 Score_c = F1 score for class c. We calculate all the values above the confusion matrix for our six classes.

		Predicted Class					
		Class 1	Class 2	Class 3	Class 4	Class 5	Class 6
Actual Class	Class 1	TN	TN	FP	TN	TN	TN
	Class 2	TN	TN	FP	TN	TN	TN
	Class 3	FN	FN	TP	FN	FN	FN
	Class 4	TN	TN	FP	TN	TN	TN
	Class 5	TN	TN	FP	TN	TN	TN
	Class 6	TN	TN	FP	TN	TN	TN

Fig. 5: Confusion matrices for class 3.

Fig. 5, shows the confusion matrices for class number 3. Following this we calculate accuracy, recall, F1 score, and precision. for our other classes.

TABLE II. HYPERPARAMETER VALUES' RANGES FOR VARIOUS ML MODELS

Model	Hyperparameter Value Range	Optimized value
XG Boost	n_estimators: [100, 200, 300], max_depth: [3, 4, 5], learning_rate: [0.01, 0.1, 0.2], subsample: [0.7, 0.8, 0.9], colsample_bytree: [0.7, 0.8, 0.9], objective: ['binary:logistic', 'multi:softmax'],	subsample: 0.9, objective: 'multi:softmax', n_estimators: 300, max_depth: 5, learning_rate: 0.1, colsample_bytree: 0.9
Random Forest	n_estimators: [100, 200, 300], max_depth: [None, 10, 20, 30], min_samples_split: [2, 5, 10], min_samples_leaf: [1, 2, 4], criterion: ['gini', 'entropy']	n_estimator: 200, min_samples_split: 5, min_samples_leaf: 1, max_depth: None, criterion: 'entropy'
Logistic Regression	'C': [0.001, 0.01, 0.1, 1, 10], 'penalty': ['l1', 'l2'], 'solver': ['liblinear', 'newton-cg', 'lbfgs', 'saga']	'C': 10, 'penalty': 'l1' 'solver': 'liblinear'

Decision Tree	max_features: [auto, sqrt, log2], max_depth: np.linspace(20, 1000, 50), min_samples_split: [2, 5, 10, 14], min_samples_leaf: [1, 2, 4, 6, 8], criterion: [entropy, gini]	max_features: [sqrt], max_depth: 520, min_samples_split: 10, min_samples_leaf: 4, criterion: entropy
KNN	p: [1,2], n_neighbors: [1-30], leaf_size: [1-50]	p: 1, n_neighbors: 1, leaf_size: 1
AdaBoost	n_estimators: [50, 200, 500], Learning_rate: [0.01, 0.1, 1.0]	n_estimators: 200, learning_rate: 0.01

Table II illustrates the hyperparameter values' ranges for all the ML models.

TABLE III. PERFORMANCE METRICS OF VARIOUS CLASSIFIERS FOR DEFAULT PARAMETERS

Classifier	Precision	Recall	F1 Score	Accuracy
XG Boost	99%	99%	99%	99.39%
Random Forest	98%	98%	98%	98.23%
Logistic Regression	99%	99%	99%	99%
Decision Tree	94%	94%	94%	94%
KNN	96%	96%	96%	95.93%
AdaBoost	88%	80%	75%	80.16%

Table III illustrates the performance matrices of various classifiers we used for default parameters. As per this table, XGBoost outperforms all the machine learning models with the highest accuracy of 99.39 percent and Decision Tree has the lowest accuracy of 80.16 percent.

TABLE IV. PERFORMANCE METRICS OF VARIOUS CLASSIFIERS WITH OPTIMIZED HYPERPARAMETERS

Classifier	Precision	Recall	F1 Score	Accuracy
XG Boost	100%	100%	100%	99.52%
Random Forest	99%	99%	99%	98.64%
Logistic Regression	99%	99%	99%	99%
Decision Tree	94%	94%	94%	93.55%
KNN	98%	98%	98%	98.31%
AdaBoost	97%	97%	97%	96.61%

Table IV, we see performance metrics of various classifiers with optimized hyperparameters. Here eight models are used. As per this table, XGBoost outperforms all the machine learning models with the highest accuracy of 99.52 percent and Decision Tree has the lowest accuracy of 93.55 percent.

Accuracy (With Default Hyper Parameter)

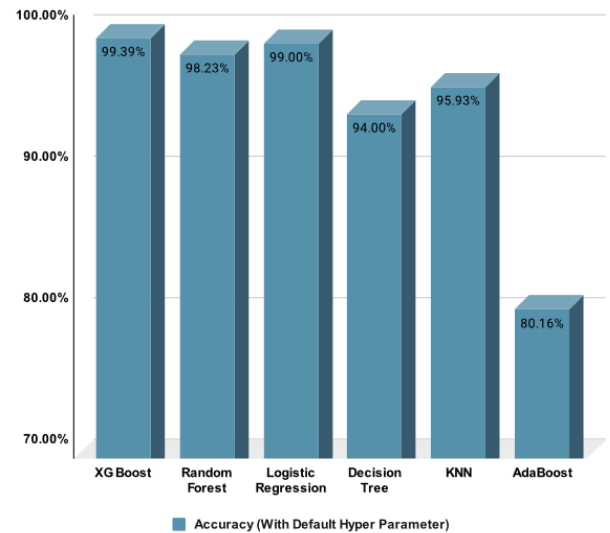


Fig. 6: Accuracy of all ML models with default hyperparameters.

Fig. 6 shows the accuracy of the ML models in the form of a bar graph with the default parameters technique. It shows that XGBoost has the best accuracy of 99.39% for the dataset and AdaBoost has the lowest accuracy of 80.16%.

Accuracy (After Hyper Parameter Tuning)

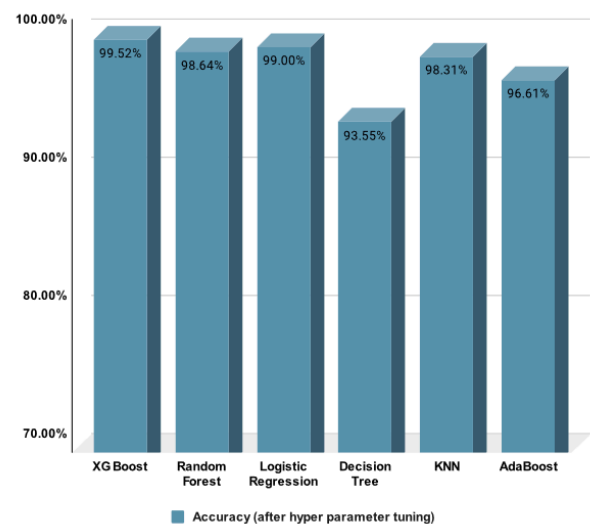


Fig. 7: Accuracy of all ML models with optimized hyperparameters using GridSearchCV or RandomSearchCV.

Fig. 7 shows the accuracy of the ML models in the form of a bar graph with optimized hyperparameters using GridSearchCV or RandomSearchCV. It shows that XGBoost has the best accuracy of 99.52% for the dataset and Decision Tree has the lowest accuracy of 93.55%.

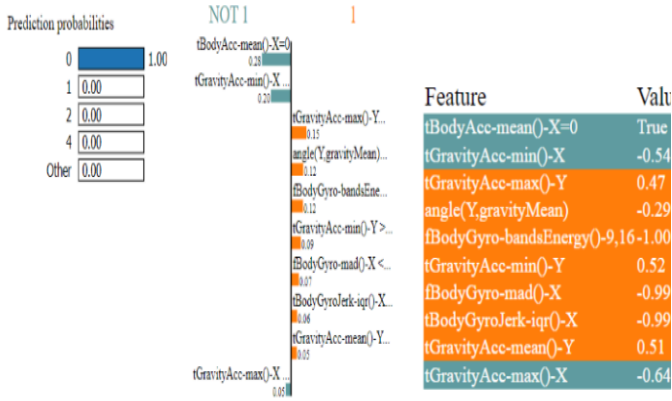


Fig. 8: Machine learning model prediction interpretation by LIME explainable AI library.

In Fig. 8, Using the LIME explainable AI framework, the logistic regression model's prediction interpretation is displayed in Fig. 5. Fig. 4 shows that, after hyperparameter optimization, the XG Boost model performs the best. The LIME prediction results are produced using this machine-learning model. The XG Boost model predicted human activity with an accuracy of 99.52%, as shown by the LIME figure. Based on tGravityAcc-max()-Y of 15% , angle(Y,gravityMean) of 12%, fBodyGyro-bandsEne 12%, tGravityAcc-min()-Y greater than 9%, fBodyGyro-mad()-X less than 7%, tBodyGyroJerk-iqr()-X 6%, tGravityAcc-mean()-Y 5%, the human activities are predicted for everyone when the value of tBodyAcc-mean()-X=0, tGravityAcc-min()-X, tGravityAcc-max()-X are True, -54%, -64% respectively.

TABLE V. COMPARISON BETWEEN OTHER PAPERS AND OUR WORKS

References	Dataset	Model	Accuracy	Recall
[3]	Private	K-mean and ANN	98.24%	N/A
[4]	MHEALTH	RNN (GPU, SRU)	99.80%	N/A
[5]	NTU RGB+D, SBU Interaction, UT-Kinect, Berkeley MHAD	HBNN	100%	N/A
[6]	UNI-HAR	ResNet	97.6%	N/A
[7]	Public	Feed-forward, KNNs, SVM, Discriminant analysis, and Naive Bayes	98%	100%
[8]	Public (PPG data collected from the physionet)	Random Forest	97.8%	N/A
[9]	Private	R.F, N.V	99.55%	N/A
[10]	Public (UCI ML)	SVM, Naive Bayes	96.78%	N/A
This work	Human Activity Recognition Using Smartphone	DT, RF, LR, KNN, XG Boost, AdaBoost	99.52%	100%

Table V: This table shows the existing work on the Human Recognition System. We describe various papers and show dataset type, ml model, accuracy, and recall. In reference 3, they use a private dataset, k-mean, and ANN model. The accuracy is 98.24%. In reference 4, they use the MHEALTH dataset and RNN (GPU, SRU) model. The accuracy is 99.80%. In reference 5, they use the NTURGB+D dataset, the HBRNN model. The accuracy is 100%. We also describe five more papers in detail. Then, compare all the criteria with this work. Here, we use Human Activity Recognition on the UsingSmartPhone dataset and DT, RF, LR, KNN, XGBOOST, and AdaBoost Models. The accuracy is 99.52%, and the recall value is 100.

IV. COMPARATIVE ANALYSIS

We compared different ML models in our “Human Activity Recognition Using Smartphones ‘public dataset. We have used Decision Tree, Logistic regression, KNN, Ransom Forest, XGBoost, and AdaBoost.

A. Accuracy Comparison

Every model's accuracy was evaluated and contrasted across several human endeavors. The results of wearable device data show that XGBoost scored the best overall accuracy, indicating its efficacy in properly identifying human actions. And its accuracy was 99.52%.

With Default hyperparameter tuning: The models vary in accuracy, with XGBoost leading at 99.39%, followed closely by Logistic Regression and Random Forest at 99%. AdaBoost falls behind at 80.16%, while Decision Tree and

KNN perform well but are slightly less accurate at 94% and 95.93%, respectively. The differences in accuracy can be attributed to each algorithm's inherent strengths and weaknesses. Because of its ensemble method, XG Boost may be particularly good at identifying intricate patterns in the data, which could explain some of its improved performance. Logistic Regression and Random Forest are robust algorithms suitable for this task, demonstrating consistent and high accuracy. Despite its integrity, Decision Tree and KNN could have trouble with some of the dataset's complexities. AdaBoost provides an ensemble approach, but its accuracy appears to be poorer because of its less successful ability to capture the subtleties of the data. The final decision on which model is best depends on the particulars of the task at hand, the dataset, and its subtleties.

After hyperparameter tuning: After hyperparameter tuning, the models exhibit notable enhancements in accuracy. XG Boost maintains its lead with a refined accuracy of 99.52%, showcasing its resilience and adaptability. Random Forest, Logistic Regression, and KNN also demonstrate improvements, solidifying their robust performance. Decision Tree remains competitive while showing a slight decrease in accuracy to 93.55%. The most significant improvement is observed in AdaBoost, which undergoes a substantial accuracy boost from 80.16% to 96.61%. These outcomes underscore the pivotal role of hyperparameter tuning in optimizing model performance across various classifiers, contributing to more accurate and reliable predictions. The comparison emphasizes the model-specific responses to hyperparameter tuning, highlighting the importance of tailoring model configurations to achieve optimal results. The preference of the most appropriate model depends on the specific characteristics and requirements of the dataset, with XG Boost continuing to stand out as the overall best performer. The nuanced responses of each model to hyperparameter tuning underscore the need for a tailored approach in selecting the optimal configuration for specific machine learning tasks.

B. Precision, Recall, and F1 Score Analysis

The models were evaluated for their capacity to reduce false positives and false negatives using precision, recall, and F1 scores. While XG Boost showed remarkable recall for 100% value, it also displayed greater precision with a value of 100% and an F1-score of 100%.

C. Runtime Comparison

When the runtime is examined, it is anticipated that a given ML approach would take longer to run on a dataset with more data (in terms of features and data gathered). The models took a good amount of time to run in both Google Colab and Jupyter Notebook 6.4.12. However, we saw that the AdaBoost model took the longest and the Decision Tree model took the shortest time to run. We also observed that Jupyter Notebook took less time than Google Colab to run all models in our "Human Activity Recognition with SmartPhones" dataset of Kaggle. If we list all the models according to run time, the decision tree takes the least time, as do random forest, KNN, etc.

V. CONCLUSIONS

This research delves into the realm of Human Activity Recognition (HAR) by extensively exploring machine

learning models applied to data from wearable devices. The study employs diverse classifiers, including XG Boost, Random Forest, Logistic Regression, Decision Tree, KNN, and AdaBoost, assessing their performance under default and hyperparameter-tuned settings. The results showcase XG Boost's superior performance, emphasizing its adaptability and resilience. However, the study also highlights the nuanced strengths of other models, such as the robustness of Logistic Regression and Random Forest and the competitive accuracy of Decision Tree and KNN. The significance of hyperparameter tuning is underscored, with notable improvements observed across models, particularly in the case of AdaBoost.

Moreover, the research integrates Explainable AI, utilizing LIME to interpret the predictions of the logistic regression model. This emphasis on interpretability contributes to the broader learning of how machine learning models arrive at their predictions. The study concludes by emphasizing the need for tailored approaches in configuring models, acknowledging the specific nuances of datasets, and recognizing the ongoing refinement required for optimal model performance. Overall, this work advances the understanding of HAR and provides valuable insights into the intricate interplay between machine learning models, hyperparameter tuning, and interpretability in the context of wearable device-centric human activity recognition.

In our research on "A Comparative Analysis of Different ML Models for Wearable Devices Centric Human Activity Recognition," we plan to refine feature engineering techniques for extracting relevant information from sensor data. Exploring advanced model architectures, including recurrent neural networks (RNNs), will enhance recognition accuracy. We aim to conduct an extensive hyperparameter search and dataset expansion to evaluate model generalization comprehensively. Integrating user feedback and ensuring ethical considerations guide our future work for responsible and personalized deployment of activity recognition models.

REFERENCES

- [1] C. Jobanputra, J. Bavishi, and N. Doshi, "Human Activity Recognition: A Survey," *Procedia Computer Science*, vol. 155, pp. 698–703, 2019.
- [2] Y. Zhao, "Deep Residual BiDR-LSTM for human activity recognition using wearable sensors," *arXiv.org*, Aug. 22, 2017.
- [3] S. Mahajan and H. Amhia, "Improvement of Video Human Activity Recognition by using ANN Approach," *International Conference on Communication and Signal Processing* 2019, pp. 0829–0832.
- [4] A. Gumaei, M. M. Hassan, A. Alelaiwi and H. Alsalman, "A Hybrid Deep Learning Model for Human Activity Recognition Using Multimodal Body Sensing Data," *IEEE Access*, vol. 7, pp. 99152–99160, 2019.
- [5] J. Liu, A. Shahroury, D. Xu, and G. Wang, "Spatio-temporal LSTM with trust gates for 3D human action recognition," *arXiv.org*, <https://arxiv.org/abs/1607.07043>
- [6] F. M. Calatrava-Nicolás and O. M. Mozos, "Light Residual Network for Human Activity Recognition using Wearable Sensor Data," in *IEEE Sensors Letters*, vol. 7, pp. 1–4.
- [7] S. Ghazal, U. S. Khan, M. Mubasher Saleem, N. Rashid, and J. Iqbal, "Human activity recognition using 2D skeleton data and supervised machine learning," *IET Image Processing*, vol. 13, pp. 2572–2578, 2019.
- [8] N. S. Pathan, Md. T. F. Talukdar, M. Quamruzzaman, and S. A. Fattah, "A Machine Learning based Human Activity Recognition during Physical Exercise using Wavelet Packet Transform of PPG and Inertial Sensors data," *IEEE Xplore*, 2019.
- [9] Zameer, Gulzar, and Leema Anny, "Human Activity Analysis Using Machine Learning Classification Techniques," *International Journal of Innovative Technology and Exploring Engineering*, vol. 9, no. 2, 10 Dec. 2019, pp. 3252–3258.
- [10] Kumar, Manoj, et al. "Dimensionality Reduction-Based Discriminatory Classification of Human Activity Recognition Using Machine Learning." *Lecture Notes in Networks and Systems*, 3 July 2022, pp. 581–593

- [11] Nurwulan, N.R. and Selamaj, G. (2021) "Human daily activities recognition using decision tree", *Journal of Physics: Conference Series*, 1833(1), p. 012039.
- [12] L. Dong et al., "Very High Resolution Remote Sensing Imagery Classification Using a Fusion of Random Forest and Deep Learning Technique—Subtropical Area for Example," in *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, vol. 13, pp. 113-128, 2020.
- [13] Peng, Chao-Ying Joanne, et al. "An Introduction to Logistic Regression Analysis and Reporting." *The Journal of Educational Research*, vol. 96, no. 1, Sept. 2002, pp. 3–14
- [14] M. Zubair, K. Song and C. Yoon, "Human activity recognition using wearable accelerometer sensors," *International Conference on Consumer Electronics-Asia*, 2016, pp. 1-5
- [15] P. Wang, Y. Zhang, and W. Jiang, "Application of K-Nearest Neighbor (KNN) Algorithm for Human Action Recognition," *Advanced Information Management, Communicates, Electronic and Automation Control Conference*, 2021, pp. 492-496.
- [16] A. Prakash, J. Thangaraj, S. Roy, S. Srivastav and J. K. Mishra, "Model-Aware XGBoost Method Towards Optimum Performance of Flexible Distributed Raman Amplifier," in *IEEE Photonics Journal*, vol. 15, no. 4, pp. 1-10, Aug. 2023, Art no. 8800210.
- [17] M. Belgiu and L. Drăguț, "Random forest in remote sensing: A review of applications and future directions", *ISPRS J. Photogramm. Remote Sens.*, vol. 114, pp. 24-31, 2016.